Astronomy & Astrophysics manuscript no. output
March 11, 2026
©ESO 2026

Predicting spin and inclination of black holes using neural networks

Report to lecture 'computational astrophysics' by Dr. Christian Fromm

Yanick Thurn

Submitted March 11, 2026

ABSTRACT

Context. Machine learning has proven to be extremely effective in image processing over the past few years. In this context, more specific neural networks.
Aims. The aim of this article is to develop two neural networks for determining the spin and inclination from images of black holes.
Methods. To this end, GRMHD simulations are used to create artificial training and test data. Furthermore, two different architectures, considering the requirements for the respective tasks, for the networks were developed.
Results. The networks were successfully trained to predict the spin up to an absolute mean error of $(3.41 \pm 0.38) \cdot 10^{-2}$ and the inclination to $(4.14 \pm 0.77)^\circ$. Thereby, the uncertainty, considered over the individual steps of the simulation data, was always below the step size of the simulation.

1. Introduction

Modern astrophysics has the problem of being able to accurately break down the processes of the smallest particles in very distant objects. However, it is only possible to record the radiation of such processes on earth and in the surrounding orbits using satellites. To be able to assign these subsequently to certain events and astronomical objects, a far-reaching analysis of these data is often inevitable. Thereby, the spectral properties of the objects are investigated and the development process is reverse engineered. Unfortunately, these often very complex processes cannot be described by simple analytical means. Therefore, in modern astrophysics numerical calculations and complex simulations are often used to mimic these processes and determine the properties of the distant objects.
One class of these objects is black holes. These were first proposed mathematically with Einstein's general theory of relativity and recently the first-ever made image of a black hole was published Collaboration (2019). One of the most studied black holes is Sagittarius A*, which is located in the center of the milky way. It is a nearby source with low luminosity.
The most prominent type of simulation is the general-relativistic magnetohydrodynamic simulation or GRMHD for short. These simulate the movement of light close to the black hole. However, determining the properties of the black hole using simulations can be time-consuming. This is where another tool of modern computer technology can help. This tool is called machine learning. One model for machine learning is the so-called neural network. These are modeled on the organic brain and have been very successful in classifying and evaluating visual data. Therefore, the question naturally arises to what extent and to what precision they can be used to directly read out the properties from the measured images. In the following, as a proof of concept, such a network will be trained to successfully predict the spin of the central black holes. However, simulation data are used first, since they are not subject to measurement fluctuations.

2. Black hole

Like every other astronomic object, black holes attract surrounding mass to themselves. As the potential energy of these masses is released when falling towards the black hole and is thermally excited. Due to its large mass and thus strong gravitational force, these energies are pretty high in the case of black holes. As the rotational momentum of the accreting mass is preserved it forms a disc around the black hole and due to the high excitation, this disc becomes plasma. Finally, through thermal processes, these discs emit radiation, whose movement is influenced by the gravitational forces of the black hole.

3. Simulation

One model to simulate this behavior is GRMHD simulations. The simulation consists of a ray tracer, which tracks the radiation generated by the disk in a relativistic framework. However, this numerical tracking of the radiation is resource-intensive. Together with the properties of the system, among others spin, inclination, and disk width, images of the black holes can be generated and finally compared with the measured images. From the similarities of these images, the properties of the system can be deduced.

4. Neural networks

Artificial intelligence is probably the most prominent field of computer science. However, it is often confused with the related field of machine learning. But artificial intelligence does not necessarily have the ability to improve itself Russell & Norvig (2016). Therefore, while machine learning is a field of artificial intelligence, not all algorithms with artificial intelligence belong to the field of machine learning.

Three different types of machine learning are known.

- The first, and most intuitive, is supervised learning. For this method to be used, a set of training data $\mathcal{D}$ must be given. The set consists of pairs (x, y), with x being the input and y being the expected output. Afterward, the model is understood as a function f with several parameters W. The learning is described by an adjustment of the parameters in such a way, that $f : x \rightarrow y$ becomes true.
- The second approach uses less information. Instead of knowing the 'perfect' solution to a given situation s_{init}, the data set consists of a 4-tuple containing the former state s_{init}, the action a, which was taken, the final state s_{final}, and an indicator how 'good' the performed action was, which is called the reward r. The 4-tuple reads $(s_{\text{init}}, a, s_{\text{final}}, r)$. While learning, the model tries to maximize its total reward.
- The third, and last approach uses the least knowledge about the outcome. Instead, the set consists only of inputs x. While training the model learns to group them. Therefore, this approach is mostly used for clustering or exploring deep features within the data.

As the properties of the black hole are known, the outcome of the model to an image of it is known. Therefore, the supervised approach will be used in the following.

As a model, neural networks are used. A neural network is a network of repeating units called neurons Mehta et al. (2018). These are grouped in layers. Within each layer, the neurons are independent of each other but dependent on all the previous ones. Thereby, each neuron receives signals from the neurons in the previous layer. These signals are weighted and finally summarised. Based on this value, the neuron itself decides if and how strong it sends a signal to the following ones. Mathematically, this is given as

$$y = \sigma(\boldsymbol{W} \cdot \boldsymbol{x} + b), \tag{1}$$

with y being the signal the neuron transits, $\boldsymbol{W}$ and $\boldsymbol{x}$ being vectors containing the weights to and the signals of the previous neurons, σ the activation function, and b a scalar called bias.

As the neurons within a layer are independent this can be done simultaneously for the whole layer. To this end, the vector $\boldsymbol{W}$ is replaced by a $N \times M$ dimensional matrix, where N and M, describe the number of neurons in the previous and the actual layer. Furthermore, the scalar bias b is promoted to a vector $\boldsymbol{b}$ holding the bias for each neuron in the layer. The activation of the neurons is then given by the components of the vector. For the layers, a distinction is made between the three categories. The first layer, to which the input for the network is applied, is referred to as the input layer. Analogously, the last layer is called the output layer. Since the layers in between do not interact with the outside world and are thus 'hidden' from it, they are called hidden layers.

The activation function plays a particularly important role. Since the connections between the neurons are linear, the network is, without an activation function, only able to learn linear functions. The use of non-linear activation functions enables them to learn arbitrary functions. Therefore, they are also called universal function approximators. These activation functions can be divided into two major groups. The convergent activation functions, including Tanh, and the divergent activation functions, e.g. ReLu. However, since the learning of the networks is based on gradient descent, it is advantageous if they have non-vanishing gradients. Therefore, Relu is often replaced by leaky Relu or other variations. The most common activation functions are shown in fig. 1. The upper row consists of convergent activation functions while the lower one shows divergent ones.

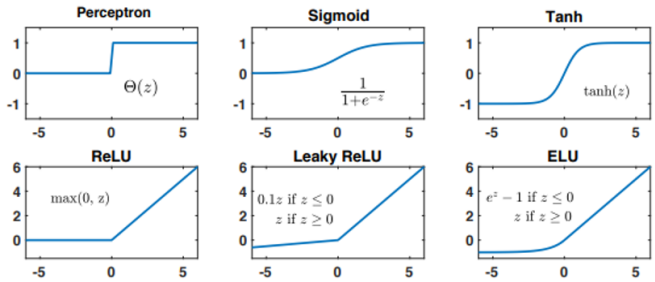

Fig. 1. The plot shows the most common activation functions for neurons in neural networks. The upper row contains only convergent, while the lower row contains divergent functions. The image was taken from (Mehta et al. 2018, p. 48,fig. 35)

5. Implementation of the network

Determining the spin and the inclination are two different tasks. Therefore, two different networks are used to do so.

The spin a is restricted to an interval $a \in [-1, 1]$. Therefore, the Tanh function can be used as an activation function in the last layer. Furthermore, the spin is encoded within the whole image. Therefore the network doesn't need to recognize single features in the image, but the network has to process the image as a whole. Therefore convolutional layers are probably less advisable here. They could identify individual features, but the kernel matrix would have to be too large for their use to be useful. This situation would change abruptly, however, if the images were not taken from simulations but from real images. In this case, small convolutional layers could be used to correct errors and deviations caused by the measuring process or other disturbances. In addition, the network should be given the option of scaling the data as required for further evaluation. Since no negative values are expected, the first layer after the input should be a ReLu or leaky ReLu function. All together now results in the structure shown in fig. 2 a). The circles represent the different layers, the first line within them the activation function, and the second line the number of neurons in the layer. The red and blue circles denote the input and output layer, respectively. The batch size is set to 64.

Since in the case of spin, the deviation between the expected result y and the result predicted by the network $\hat{y}$ can have a maximum 2 but will mostly be smaller than 1, the mean absolute error, sometimes also called the L1 loss, was used as loss $\mathcal{L}$. Mathematically, this can be described as follows

$$\mathcal{L} = \sum_{x \in \mathcal{D}} |y - \hat{y}(x)|, \tag{2}$$

where $\mathcal{D}$ denotes the training data set.

For the inclination, the situation is slightly different. While the general structure remains, meaning the size of the different layers, the activation functions change. The inclination i is restricted to an interval $i \in [0, 180]$. An important aspect is that the values are all positive. Therefore, the Tanh layers are

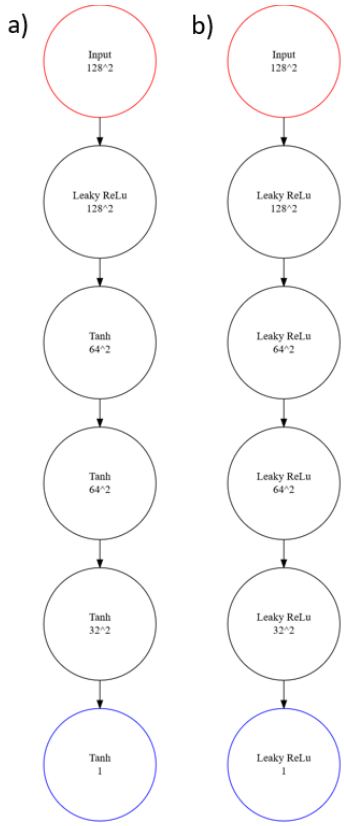

Fig. 2. Scheme of the spin-network a) and the inclination network b). The circles represent the different layers, the first line within them the activation function and the second line the number of neurons in the layer. The red and blue circles denote the input and output layer, respectively.

replaced by leaky ReLu layers. The structure is shown in fig. 2 b). The batch size is set to 32.

In the case of the inclination, the range of the deviation is quite large. Therefore, as loss, the mean square error is used. This is given as

$$\mathcal{L} = \sum_{x \in \mathcal{D}} (y - \hat{y}(x))^2 \,. \tag{3}$$

6. Results

Using the simulation, 8075 images of black holes, one for each permutation of the parameters listed in the tab. 1, are generated. Afterward, 10%, in absolute numbers 808, of these were randomly selected as test data. Using the other 7267 data, the networks described above are trained.

	a	i	kv	h
Interval	[-0.9, 0.9]	[10, 170]	[0.2, 1]	[0.2, 1]
Step	0.1	10	0.2	0.2

Table 1. Parameters that are varied during the simulation. For each permutation of the parameters, one image was generated.

The training consisted of a different number of epochs with different learning rates. The spin network was first trained 5 epochs with a learning rate of $lr = 1e-4$ and afterward 20 epochs with a learning rate of $lr = 5e-5$. Each set of 5 epochs of training is marked with a different color. The loss is monitored while training. It is calculated using the test data. It can be seen in fig. 3. The loss before the first training epoch is not shown in the plot. Therefore, epoch=0 describes the loss after the first epoch.

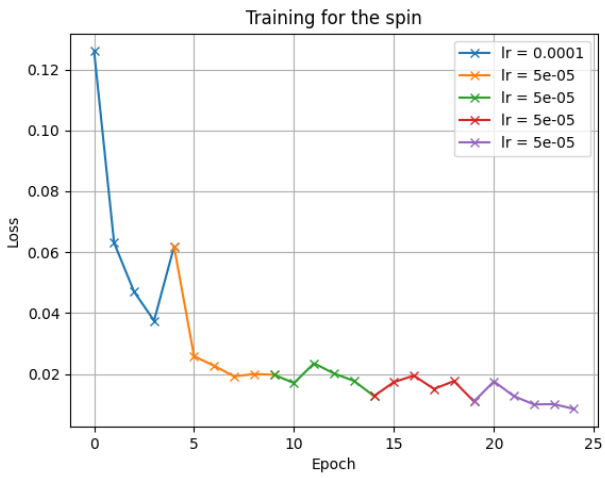

Fig. 3. Loss over the different training epochs for the spin network. The loss for the untrained network is not shown in the graph. Therefore, the loss at epoch 0 describes the loss after the first epoch. The training was done with two different learning rates lr. Each set of 5 epochs of training is marked with a different color.

Analog to the spin network, the inclination network was trained. Its loss can be seen in fig. 4. At first, the network was trained 5 epochs with a learning rate of $lr = 1e-3$ followed by 2 epochs with $lr = 5e-4$. As the loss is not yet raising again, the network is not starting to overfit. Furthermore, the behavior of the loss function, namely the mean square error, must be considered. Since this is parabolic, the flattening at the end of epoch 7 does not necessarily indicate stagnation but should be an artifact of the loss function.

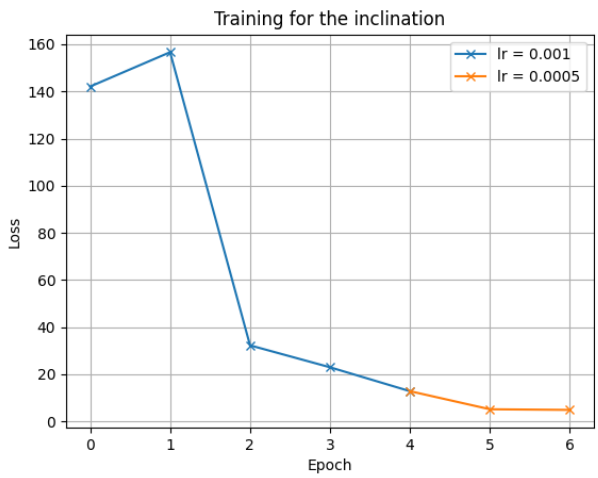

Fig. 4. Loss over the different training epochs for the inclination network. The loss for the untrained network is not shown in the graph. Therefore, the loss at epoch 0 describes the loss after the first epoch. The training was done with two different learning rates lr.

Using both networks, the spin and inclination of the test data could be identified. Thereby, the mean absolute deviance was determined to be

- $(8.39 \pm 0.63) \cdot 10^{-3}$ for the spin network and
- $(4.14 \pm 0.77)^\circ$ for the inclination network.

An example of an image and the corresponding classification is given in fig. 5. The true and estimated spin match. The difference in inclination is 3°. Therefore, the inclination is compatible with the determined deviance within its range of uncertainty.

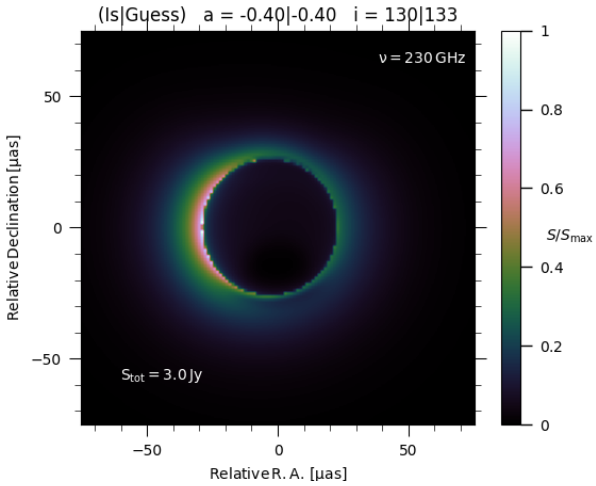

Fig. 5. Example image for classification. The networks assign a spin of −0.4 and an inclination of 133° to the black hole shown. Therefore, the estimated inclination and the true inclination differ 3°.

Finally, to get a deeper insight into the uncertainty of the predictions, the mean absolute error depending on the expected spin is determined. To this end, the test data were considered and grouped by their actual spin. Afterward, the predictions of the spin network were consulted and compared to the expected ones. The results of this comparison can be seen in fig. 6. It can be seen that the mean error is highest in the range around −0.8. However, even in this case the uncertainty is around 0.017 and thus smaller than the steps in the training data of 0.1. Analogue was done for the inclination. The result can be seen in fig. 7. The uncertainty increases with the size of the inclination. However, it is obvious that the relative error becomes significantly smaller with increasing inclination. Around the largest inclination of 180°, the uncertainty is barely 3° and is thus about one-third of the step size of 10° for the inclination.

For a more intuitive approach to the results, the confusion matrices for spin and inclination are calculated. Since these are normally used in classification problems, but the networks output continuous values for spin and inclination, these must be binned. The steps of the training data are used in different categories. The outputs of the network are assigned to the closest category. The confusion matrices can be seen in fig. 8 and 9, respectively. As expected, the main part of the data classifications is on the diagonal. Therefore, both networks learned the training data well.

7. Conclusion

To determine the spin and the inclination of black holes using machine learning, two different network architectures were developed. Using the GRMHD simulations these networks were

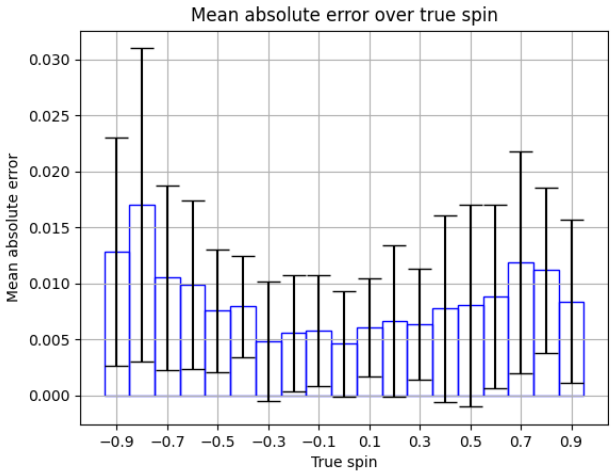

Fig. 6. Uncertainty, represented by the mean absolute error over the actual spin. A peak at −0.8 can be seen. But even there, the mean absolute error is smaller than the step size of 0.1 in the training data. The black bars mark the sample standard deviation.

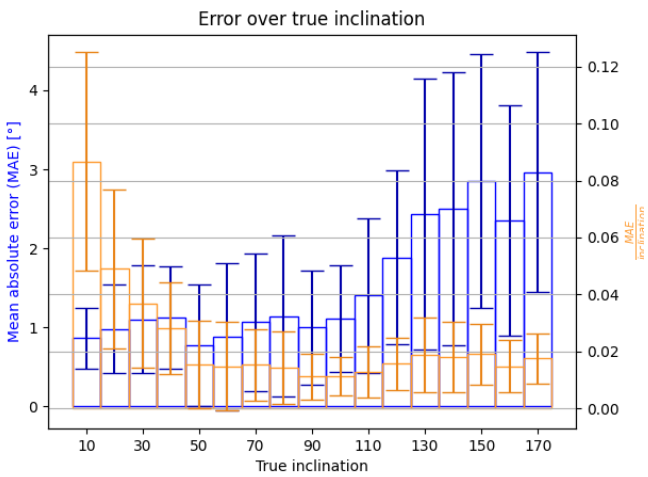

Fig. 7. Uncertainty, represented by the error over the actual inclination. While the absolute error (blue) increases with increasing inclination, the relative (orange) decreases. The dark blue, dark orange bars mark the sample standard deviation.

successfully trained to determine the spin and inclination of the black holes. Thereby, the spin and inclination networks had an average deviation of $(3.41 \pm 0.38) \cdot 10^{-2}$ and $(4.14 \pm 0.77)^\circ$, respectively, over the test data. The development of the loss over the training shows that, at least in the case of the inclination, a further improvement could be achieved with more training and possibly changed hyperparameters, especially the learning rate. Nevertheless, a network for determining the properties of a black hole from the images was successfully trained. In the next step, these networks could now be tested on real data. For this, however, as mentioned above, convolutional layers should be used to eliminate uncertainties and fluctuations caused by the measurement. Furthermore, the step size used for the simulation is a limitation for learning. To further increase the accuracy of the network, stochastic values from a continuous spectrum could be chosen for the parameters, and/or a higher number of training data, with a smaller step size, could be generated.

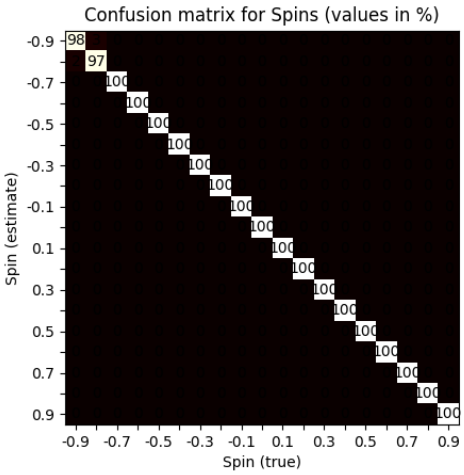

Fig. 8. Confusion matrix for the spins. The different categories are given by the step size of the training data. The values are the relative number of predicted spins for the true spin in percent.

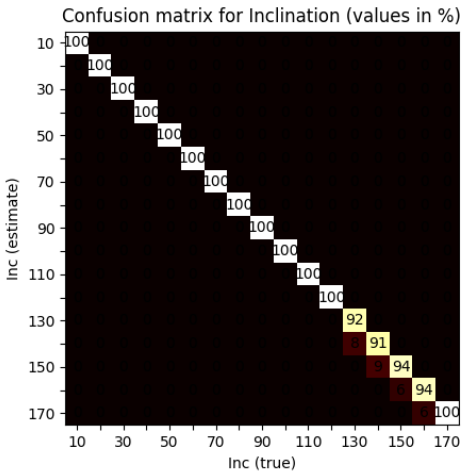

Fig. 9. Confusion matrix for the inclination. The different categories are given by the step size of the training data. The values are the relative number of predicted inclinations for the true inclination in percent.

References

Collaboration T.E.H.T., 2019
Mehta P., Bukov M., Wang C.H., et al., 2018
Russell S., Norvig P., 2016, Artificial Intelligence - A Modern Approach, Pearson, München

Appendix A: Code: Dataloader

```
from typing import List, Union, Tuple
import torch
import numpy as np
import re

"""
functions to extract:
-> Correct spin
-> Correct inclination
from the filename
Example filename:
-> a-0.00_inc010_kv0.20_h1.00_GRRT.npz
"""
functions = {
    "a": (lambda x: float(re.findall("a-?\d+.?\d*", x)[0].replace("a", ""))),
    "inc": (lambda x: float(re.findall("inc\d+", x)[0].replace("inc", "")))
}


class DataGen:
    """
    Iterable for the train and test data
    The data are loaded not all at once to save RAM
    """

    def __init__(self, files: List[str], target: str, bs: int = 64, path: str = "",
                 device: Union[str, torch.device] = "cpu"):
        """
        Initialize a data Iterator
        :param files: List[str] = List of filenames with the image data
        :param target: str = either 'a' or 'inc' for spin or inclination, respectively
        :param bs: int = batch size
        :param path: str = path to the files
        :param device: Union[str, torch.device] =  device to put the tensors on
        """
        self.f = functions[target]
        self.bs = bs
        self.path = path
        self.device = device

        self.files = files
        self.length = len(self.files)

        self.images = torch.empty((self.bs, 128 ** 2))
        self.targets = torch.empty((self.bs, 1))

    def __iter__(self) -> Tuple[torch.tensor, torch.tensor]:
        """
        Iterate over the DataGen object
        :return: yield Tuple[torch.tensor, torch.tensor] = (input_data, target_data)
        """
        c = 0
        for file in self.files:
            if c == self.bs:
                yield self.images.to(self.device), self.targets.to(self.device)
                c = 0

            self.images[c, :] = torch.from_numpy(np.load(f"{self.path}/{file}")["image"].flatten
                ()).float()
            self.targets[c] = self.f(file)
            c += 1

        # necessary since set may not be exactly dividable by bs
        # in this case some data will occur twice (filling the tensor to bs)
        yield self.images.to(self.device), self.targets.to(self.device)
```

Appendix B: Code: Networks

```python
from typing import List, Union, Iterable
import torch
import time


class SpinNetwork:
    """
    Basis class for the network which tries to determine the spin
    """

    def __init__(self, device: Union[str, torch.device] = "cpu"):
        """
        Initialize the network
        :param device: Union[str, torch.device] = device to do the calculations on
        """
        self.device = device
        self.loss = torch.nn.L1Loss()

        self.model = torch.nn.Sequential(
            torch.nn.Linear(128 ** 2, 128 ** 2),
            torch.nn.LeakyReLU(),
            torch.nn.Linear(128 ** 2, 64 ** 2),
            torch.nn.Tanh(),
            torch.nn.Linear(64 ** 2, 64 ** 2),
            torch.nn.Tanh(),
            torch.nn.Linear(64 ** 2, 32 ** 2),
            torch.nn.Tanh(),
            torch.nn.Linear(32 ** 2, 1),
            torch.nn.Tanh()
        )
        self.model.to(self.device)
        self.optimizer = None

    def train(self, train_data: Iterable, test_data: Iterable, its: int = 5, lr: float = 5e-3)
        -> List[float]:
        """
        Train and Test the created model
        :param train_data: Iterable = set of trainings data
        :param test_data: Iterable = set of test data
        :param its: int = number of training epochs
        :param lr: float = learning rate for Adam
        :return: List[float] = Loss for each trained epoch (determined by test data)
        """
        losses = []
        optimizer = torch.optim.Adam(self.model.parameters(), lr)
        for i in range(its):
            self.model.train()
            t1 = time.time()
            for inp, out in train_data:
                loss = self.loss(self.model(inp), out)

                loss.backward()
                optimizer.step()
                optimizer.zero_grad()

            loss_epoch = self.eval(test_data)
            losses.append(sum(loss_epoch) / len(loss_epoch))

            print(f"Epoche:{i}\nTime: {time.time() - t1}s\nLoss: {losses[-1]}\n\n")

        return losses

    def eval(self, test_data: Iterable) -> List[float]:
        """
        Evaluate the created network
        :param test_data: Iterable = set of test data
        :return: List[float] = Loss for given set
        """
        self.model.eval()
        with torch.no_grad():
```

```
            loss_epoch = []
            for inp, out in test_data:
                res = self.model(inp)
                loss_epoch.append(self.loss(res, out).item())

        return loss_epoch

    def predict(self, image: torch.tensor) -> float:
        """
        Predict the spin for a given image
        :param image: torch.tensor[1d] = image of the BH
        :return: float = predicted spin
        """
        self.model.eval()
        with torch.no_grad():
            res = self.model(image).item()
        return res


class IncNetwork:
    """
    Basis class for the network which tries to determine the inclination
    """

    def __init__(self, device="cpu"):
        """
        Initialize the network
        :param device: Union[str, torch.device] = device to do the calculations on
        """
        self.device = device
        self.loss = torch.nn.MSELoss()

        self.model = torch.nn.Sequential(
            torch.nn.Linear(128 ** 2, 128 ** 2),
            torch.nn.LeakyReLU(),
            torch.nn.Linear(128 ** 2, 64 ** 2),
            torch.nn.LeakyReLU(),
            torch.nn.Linear(64 ** 2, 64 ** 2),
            torch.nn.LeakyReLU(),
            torch.nn.Linear(64 ** 2, 32 ** 2),
            torch.nn.LeakyReLU(),
            torch.nn.Linear(32 ** 2, 1),
            torch.nn.LeakyReLU()
        )
        self.model.to(self.device)
        self.optimizer = None

    def train(self, train_data: Iterable, test_data: Iterable, its: int = 5, lr: float = 5e-3)
        -> List[float]:
        """
        Train and Test the created model
        :param train_data: Iterable = set of trainings data
        :param test_data: Iterable = set of test data
        :param its: int = number of training epochs
        :param lr: float = learning rate for Adam
        :return: List[float] = Loss for each trained epoch (determined by test data)
        """
        losses = []
        optimizer = torch.optim.Adam(self.model.parameters(), lr)
        for i in range(its):
            self.model.train()
            t1 = time.time()
            for inp, out in train_data:
                loss = self.loss(self.model(inp), out)

                loss.backward()
                optimizer.step()
                optimizer.zero_grad()

            loss_epoch = self.eval(test_data)
```

```
            losses.append(sum(loss_epoch) / len(loss_epoch))

            print(f"Epoche:{i}\nTime: {time.time() - t1}s\nLoss: {losses[-1]}\n\n")

        return losses

    def eval(self, test_data: Iterable) -> List[float]:
        """
        Evaluate the created network
        :param test_data: Iterable = set of test data
        :return: List[float] = Loss for given set
        """
        self.model.eval()
        with torch.no_grad():
            loss_epoch = []
            for inp, out in test_data:
                res = self.model(inp)
                loss_epoch.append(self.loss(res, out).item())

        return loss_epoch

    def predict(self, image: torch.tensor) -> float:
        """
        Predict the spin for a given image
        :param image: torch.tensor[1d] = image of the BH
        :return: float = predicted spin
        """
        self.model.eval()
        with torch.no_grad():
            res = self.model(image).item()
        return res
```

Appendix C: Code: Training

```
"""# Seperate traing and test data """
import os
import random

path = "/content/drive/MyDrive/RIAFML"
files = [file for file in os.listdir(path) if not os.path.isdir(f"{path}/{file}")]

# create empty folders
os.mkdir(f"{path}/test")
os.mkdir(f"{path}/train")

# randomly choose 10% of the data as training data
random.shuffle(files)
test_files = files[::10]
train_files = [file for file in files if not (file in test_files)]

# sort files to correct folders
for file in test_files:
  os.rename(f"{path}/{file}", f"{path}/test/{file}")

for file in train_files:
  os.rename(f"{path}/{file}", f"{path}/train/{file}")

"""# Start Training"""

import torch
import random
import os

# check hardware
torch.cuda.empty_cache()
device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
print(f"Device: {device}")

from drive.MyDrive.CAP.network import SpinNetwork, IncNetwork
from drive.MyDrive.CAP.datagen import DataGen
```

```
TARGET = "inc"  # change to "a" to load targets for spin
path = "/content/drive/MyDrive/RIAFML"


train_files = [file for file in os.listdir(f"{path}/train") if file[-4:] == ".npz"]
random.shuffle(train_files)
train_loader = DataGen(train_files, TARGET, path=f"{path}/train", device=device, bs=32)

test_files = [file for file in os.listdir(f"{path}/test") if file[-4:] == ".npz"]
random.shuffle(test_files)
test_loader = DataGen(test_files, TARGET, path=f"{path}/test", device=device, bs=32)
print(f"Prepared {len(train_files)} train files with {len(test_files)} test files")

import matplotlib.pyplot as plt

losses = []
net = IncNetwork(device=device)  # change to SpinNetwork to train the network for spin

lr = 1e-3
loss = net.train(train_loader, test_loader, its=5, lr=lr)
losses.append((lr, loss))

plt.plot(loss)
plt.show()

torch.save(net.model.state_dict(), "drive/MyDrive/CAP/Inc_network.net")
# net.model.load_state_dict(torch.load("drive/MyDrive/CAP/Inc_network.net", map_location=device)
    )  # for loading

print(losses)
```